

Rapport Comparatif Academique

Approches de Self-Healing pour Tests Automatises Playwright

Projet: Systeme de Tests Auto-Cicatrisants pour Application Web Arcane Shop

Date: 24 June 2026

4 Strategies: Pipeline ML | Agents LLM | Selecteurs Natifs | Vision IA

Ce rapport presente une analyse comparative detaillee de quatre approches distinctes de self-healing pour tests Playwright, basees sur des donnees reelles collectees lors de l'execution de plus de 200 scenarios de test sur l'application Arcane Shop.

Table des Matieres

1. Resume Executif	3
2. Introduction & Contexte	4
3. Presentation des 4 Approches	5
4. Architecture et Mecanismes	8
5. Metriques de Performance	10
6. Tableau Comparatif	12
7. Analyse par Scenario	14
8. Forces et Limites	16
9. Recommandations	18
10. Conclusion	19
11. Annexes	20

1. Resume Executif

Ce rapport compare quatre approches distinctes de self-healing (auto-cicatrisation) pour les tests automatisés Playwright, dans le cadre d'un projet académique d'ingénierie logicielle. L'application cible est une boutique e-commerce fictive (Arcane Shop) contenant des sélecteurs HTML intentionnellement cassés (typos, transpositions) pour valider chaque approche.

Les quatre approches évaluées sont:

(1) Pipeline ML - Un système multi-étages hébergé sur Google Colab utilisant une combinaison de correction typographique, similarité sémantique (Sentence-Transformers), TF-IDF, similarité structurelle d'images (SSIM) et vision par ordinateur (EfficientNet-B0).

(2) Agents GitHub Copilot - Des agents IA (planner, generator, healer) qui planifient, génèrent et réparent les tests via l'API Copilot.

(3) Sélecteurs Natifs Playwright - Utilisation des sélecteurs résilients getByRole, getByText, getByTestId qui évitent la casse par conception.

(4) AppliTools Eyes - Comparaison visuelle par IA qui détecte les changements d'interface en comparant des captures d'écran.

Résultat principal: le pipeline ML atteint 95.3% de précision et 100% de rappel sur 43 cas de test, avec un score F1 de 0.97. Les sélecteurs natifs Playwright offrent la meilleure prévention avec 0 coût de maintenance. AppliTools excelle pour la régression visuelle. Les agents Copilot offrent une flexibilité maximale mais un coût d'exécution élevé.

2. Introduction & Contexte

2.1 Contexte du Projet

Le projet vise à évaluer différentes stratégies de self-healing pour les tests automatisés d'applications web. L'application de référence (Arcane Shop) est une boutique e-commerce riche en fonctionnalités: navigation, recherche, panier, wishlist, checkout, filtres, et modales. Environ 25% des sélecteurs dans les tests de référence sont intentionnellement altérés (typos, caractères transposés, mots manquants) pour simuler des changements de DOM et valider la capacité de chaque approche à guérir les sélecteurs cassés.

Le référentiel de tests comprend 9 fichiers couvrant:

- Structure de page (22 tests)
- Navigation (16 tests)
- Hero et bannières (16 tests)
- Fiches produits (15 tests)
- Modale produit (17 tests)
- Panier (4 tests)
- Wishlist (10 tests)
- Recherche (17 tests)
- Checkout (16 tests)

2.2 Définition du Self-Healing

Dans le contexte des tests automatisés, le self-healing désigne la capacité d'un système à détecter et corriger automatiquement les sélecteurs HTML cassés suite à des modifications de l'interface utilisateur. Les causes typiques incluent: changements de classes CSS, restructuration du DOM, mises à jour de bibliothèques frontend, ou erreurs humaines. Quatre stratégies complémentaires sont explorées ici: réactive (pipeline ML), agentic (LLM), préventive (sélecteurs natifs), et visuelle (IA).

2.3 Objectifs de l'Etude

Les objectifs de cette étude sont:

1. Évaluer quantitativement chaque approche (précision, rappel, temps).
2. Comparer les coûts d'implémentation et de maintenance.
3. Analyser les compromis entre approches réactives et préventives.
4. Fournir des recommandations pour le choix d'une stratégie selon le contexte.

3. Presentation des 4 Approches

3.1 Pipeline ML (self-healing-playwright-v2)

Architecture: Un pipeline multi-etages heberge sur Google Colab avec ngrok. Le systeme capture le DOM de la page et execute 5 etages de traitement:

- Correction typographique (difflib) - correction rapide des fautes de frappe
- Semantique (all-MiniLM-L6-v2) - similarite cosinus entre embeddings de texte
- Contextuel - scoring par balises HTML, ARIA labels, classes
- Textuel (TF-IDF) - comparaison de frequence de termes
- Visuel (SSIM + EfficientNet-B0) - similarite structurelle + features CNN

Le client Playwright intercepte les echecs via des Proxy sur les objets Page et Locator, appelle l'API Colab, verifie le selecteur gueris sur la page live, et patche automatiquement le fichier de test avec le selecteur corrige.

3.2 Agents GitHub Copilot (playwright-agents-demo)

Architecture: Trois agents IA definis dans .github/agents/:

- Planner - explore l'application et genere un plan de test en markdown
- Generator - genere le code Playwright a partir du plan
- Healer - execute les tests, analyse les echecs, propose et applique des corrections

Les agents utilisent les outils MCP (Model Context Protocol) pour interagir avec le navigateur en temps reel. L'approche est entierement basee sur LLM sans pipeline ML specialise.

3.3 Selecteurs Natifs Playwright (playwright-native-healing)

Architecture: Approche preventive utilisant les selecteurs resilients de Playwright. Les tests sont ecrits avec:

- getByRole() - selecteurs par role ARIA (boutons, liens, en-tetes)
- getByText() - selecteurs par contenu textuel
- getByTestId() - selecteurs par attributs data-testid
- getByLabel() / getByPlaceholder() - selecteurs de formulaires

Playwright fournit un auto-retry integre pour les assertions et les actions. Les selecteurs semantiques sont intrinsequement resilients car bases sur le contenu et le role plutot que sur la structure du DOM. Cette approche ne necessite aucun service externe ni pipeline ML.

3.4 Applitools Eyes (playwright-applitools)

Architecture: Solution de testing visuel par IA. Chaque test capture des screenshots de regions specifiques de l'interface, compares ensuite a des baselines via l'IA d'Applitools.

- Target.window() - capture plein ecran
- Target.region() - capture d'une zone specifique
- Detection de changements visuels (layout, couleurs, typographie)
- Mise en evidence des differences avec overlay

Le self-healing est assure par l'IA visuelle qui trouve la region correspondante meme si le layout a legerement change. Necessite une cle API Applitools et une connexion internet.

4. Architecture et Mechanismes

4.1 Pipeline ML - Flux d'Execution

Le flux d'execution du pipeline ML se decompose en 6 etapes:

1. Interception: Le Proxy Playwright capture l'echec du selecteur (TimeoutError ou echec d'assertion).
2. Appel API: Le selecteur casse est envoye au endpoint /heal du Colab.
3. Correction typographique: `difflib.get_close_matches()` cherche le plus proche match parmi tous les IDs, data-testids, classes, noms et textes du DOM. Si le score > 0.75, retour immediat (strategy='typo').
4. Pipeline ML (si typo echoue): Les 5 etages s'executent sequentiellement en funnel: tous les elements -> top-15 (semantique) -> top-8 (contextuel) -> top-10 (TF-IDF) -> ranke (SSIM) -> re-ranke (CNN).
5. Verification: Le selecteur gueris est verifie sur la page live (count > 0).
6. Patching: Le fichier de test est automatiquement mis a jour avec le selecteur corrige (backup .bak cree).

4.2 Agents Copilot - Cycle de Vie

Le cycle de vie des agents suit un processus DevOps:

1. Planification: L'agent planner explore l'URL cible avec les outils MCP navigateur, identifie les elements interactifs, et genere un plan de test.
 2. Generation: L'agent generator transforme le plan en code Playwright avec des selecteurs identifies lors de l'exploration.
 3. Execution & Healing: L'agent healer execute les tests, detecte les echecs, analyse les messages d'erreur, et propose des corrections iterativement.
- Le processus est guide par l'IA mais necessite une validation humaine.

4.3 Selecteurs Natifs - Mechanisme de Resilience

Playwright implemente plusieurs mecanismes de resilience integres:

- Auto-attente: chaque action attend automatiquement que l'element soit visible et stable avant d'interagir.
- Auto-retry: les assertions comme `toBeVisible()` retentent jusqu'a ce que la condition soit satisfaite (ou timeout).
- Selecteurs semantiques: `getByRole('button', { name: 'Submit' })` est resilient aux changements de classes, d'IDs, et de structure DOM.
- `getByTestId`: base sur des attributs dedies, immunes aux changements de styling et de restructuration.
- Chaines de locators: combinaison de `.filter()`, `.first()`, `.last()` pour affiner la selection sans selecteurs CSS fragiles.

4.4 Applitools - Mechanisme Visuel

Applitools Eyes utilise un moteur de vision par ordinateur (Visual AI) qui fonctionne differemment du matching DOM:

- Capture du rendu visuel de l'ecran (pixels).
- Comparaison avec une baseline via l'IA qui ignore les differences non significatives (anti-aliasing, polices, nuances).
- Detection des regions ajoutees/supprimees/modifiees.
- Mise en evidence visuelle des differences pour inspection humaine.
- Le self-healing vient de la capacite de l'IA a faire correspondre des regions meme avec des déplacements ou redimensionnements.

5. Metriques de Performance

Les metriques ci-dessous sont basees sur l'execution de 43 scenarios de test pour le pipeline ML, 158 tests pour les selecteurs natifs, et des donnees publiees pour Applitools et les agents Copilot.

5.1 Pipeline ML - Resultats Globaux

Metrique	Valeur
Total des Cas	43
Vrais Positifs (TP)	33
Faux Positifs (FP)	2
Vrais Negatifs (TN)	8
Faux Negatifs (FN)	0
Accuracy	95.3%
Precision	94.3%
Recall	100.0%
F1 Score	0.9706
Taux de Faux Positifs	5.7%
Taux de Faux Negatifs	0.0%
Taux de Heal	81.4%

5.2 Timing du Pipeline ML

Phase	Moyenne (ms)	Mediane (ms)	P95 (ms)	Min (ms)	Max (ms)	N
Global	47 804	57 195	65 960	478	75 575	43
Typo (diffib)	58 616	57 431	66 925	54 783	75 575	35
Pipeline ML	498	496	536	478	536	8

5.3 Distribution des Scores de Confiance

Metrique	Valeur
Moyenne	0.7839
Ecart-type	0.3755
Minimum	0.0000
25e percentile	0.9333
Mediane	0.9565
75e percentile	0.9714
Maximum	1.0000
Haute confiance (≥ 0.8)	35 cas
Confiance moyenne (0.45-0.8)	0 cas
Basse confiance (< 0.45)	8 cas

5.4 Performance par Scenario

Scenario	Accuracy	Precision	Taux FP	F1
Page Structure	100.0%	100.0%	0.0%	1.0000
Navigation	100.0%	100.0%	0.0%	1.0000
Hero & Banners	75.0%	66.7%	33.3%	0.8000
Product Modal	100.0%	100.0%	0.0%	1.0000
Cart	100.0%	100.0%	0.0%	1.0000

Self-Healing Test Automation - Comparative Academic Report

Wishlist	100.0%	100.0%	0.0%	1.0000
Search	83.3%	80.0%	20.0%	0.8889
Checkout	100.0%	100.0%	0.0%	1.0000

6. Tableau Comparatif General

6.1 Caracteristiques Techniques

Caracteristique	Pipeline ML	Agents Copilot	Selecteurs Natifs	Applitoools
Type healing	Reactif	Reactif	Preventif	Visuel
Service externe	Colab + ngrok	GitHub API	Aucun	Applitoools Cloud
Dependances	13 packages ML	Copilot + MCP	Playwright seul	eyes-playwright
Modele IA	5 stages ML	GPT-4 / Claude	N/A	Vision IA proprietare
Auto-patching	Oui (.bak backup)	Manuel	N/A	N/A
Infrastructure	Google Colab	GitHub	Local	Cloud
Cout execution	Gratuit (Colab)	~\$0.10/test	Gratuit	Payant/licence
Temps moyen/test	~48s (typo)	~30-60s	~5-15s	~10-20s
Complexite setup	Elevee	Moyenne	Faible	Faible
Maintenance	Modele + ngrok	Prompts agents	Minimale	Baselines

6.2 Metriques Comparatives

Metrique	Pipeline ML	Selecteurs Natifs	Applitoools
Accuracy	95.3%	100%*	99%+
Precision	94.3%	100%*	99%+
Recall	100.0%	100%*	98%+
F1 Score	0.9706	1.0*	~0.99
Taux FP	5.7%	0%*	~1%
Temps d'execution	~48s	~10s	~15s
Couverture defaults	Typo + DOM	Typo + structure	Visuel
Cout initial	Eleve	Faible	Moyen
Cout recurrent	Colab gratuit	Nul	Licence

* Les selecteurs natifs previennent la casse (pas de selecteurs casses).

Les metriques Applitoools sont basees sur les donnees publiees par le fournisseur.

7. Analyse Detaillee par Scenario

7.1 Structure de Page (8 cas evalues)

Le pipeline ML atteint 100% sur ce scenario. Les 7 typos sont correctement corrigees (promo-barr -> promo-bar, nav-hom -> nav-home, logoo -> logo, etc.). Un selecteur inexistant est correctement rejete (TN). Les selecteurs natifs Playwright evitent ces erreurs en utilisant getByTestId directement. Applitools verifie visuellement que la page entiere s'affiche correctement.

7.2 Navigation (5 cas evalues)

100% de succes pour le pipeline ML. Les typos sur les data-testid de navigation sont corrigees (tab-new-arrival -> tab-new-arrivals, nav-abut -> nav-about, etc.). Les selecteurs natifs utilisent getByTestId('nav-shop') qui est immunise contre les typos. Applitools capture les etats visuels apres chaque action de navigation.

7.3 Hero & Banners (4 cas evalues)

75% d'accuracy - un faux positif detecte sur le selecteur .btn-gost (corrige en .btn-ghost mais le match textuel sur 'lookbook' echoue car le parsing ne peut pas verifier le texte contenu dans un selecteur de classe compose. C'est une limitation du module de verification par parsing CSS.

7.4 Panier & Wishlist (9 cas evalues)

100% de succes pour les scenarios panier et wishlist. Les typos sur cart-panell -> cart-panel, wishlist-panell -> wishlist-panel, et wishlist-countt -> wishlist-count sont correctement corrigees. Les selecteurs natifs utilisent getByTestId('cart-panel') qui evite ces erreurs.

7.5 Recherche (6 cas evalues)

83.3% d'accuracy - un faux positif sur .search-tag -> .search-tag (le parsing de selecteur de classe ne peut pas verifier le texte contenu). Les autres typos (search-modall, search-inputt, filter-checkbox-techh) sont correctement corrigees.

7.6 Checkout (6 cas evalues)

100% de succes. Tous les selecteurs de checkout sont correctement gueris (co-subtotall -> co-subtotal, checkout-email -> checkout-email, etc.). Scenario avec le temps de healing le plus eleve (moyenne 66s typo) du a la complexite du DOM de la page checkout.

8. Forces et Limites

8.1 Pipeline ML

Forces:

- Excellent recall (100%) - ne manque aucun selecteur guerissable
- Haute precision (94.3%) - rares faux positifs
- Correction typographique efficace (81.4% des cas)
- Pipeline ML complet avec 5 etages complementaires
- Auto-patching des fichiers de test
- Systeme de verification avant patch

Limites:

- Necessite une connexion internet et un tunnel ngrok actif
- Temps de traitement eleve (~48s moyen, ~58s pour typo)
- Complexite d'infrastructure (Colab + ngrok + serveur)
- Parsing de selecteurs compose limite
- Vulnerabilite aux pannes ngrok/Colab
- Cout compute Colab (GPU pour vision)

8.2 Agents Copilot

Forces:

- Flexibilite maximale - planifie, genere et repare
- Compr hension semantique du contexte applicatif
- Guide par IA avec validation humaine
- Workflow integre dans GitHub Actions
- Documentation automatique des plans de test

Limites:

- Cout API eleve (~\$0.10 par appel)
- Lenteur (30-60s par iteration)
- Necessite une supervision humaine
- Pas de pipeline automatise sans CI/CD
- Qualite variable selon le modele utilise

8.3 Selecteurs Natifs Playwright

Forces:

- Gratuit et sans service externe
- Extremement rapide (~10s par test)
- Zero maintenance d'infrastructure
- Selecteurs immunises contre les changements DOM
- Auto-retry et auto-wait integres
- Documentation et communaute riches

Limites:

- Ne corrige pas les selecteurs existants (preventif)
- Necessite des attributs data-testid dans l'application
- Ne couvre pas les changements visuels
- Impossible si l'equipe ne suit pas les bonnes pratiques
- Pas de correction automatique des tests existants

8.4 AppliTools Eyes

Forces:

- Detection de changements visuels subtils
- IA sophistiquee pour le matching de regions
- Dashboard centralise de baselines

- Integre les tests visuels aux pipelines CI/CD
- Support multiplateforme (web, mobile, natif)

Limites:

- Cout de licence eleve
- Necessite une connexion internet
- Fausses alertes sur changements non significatifs
- Maintenance des baselines necessaire
- Ne couvre pas la logique fonctionnelle
- Apprentissage initial de l'IA

9. Recommendations

Base sur l'analyse comparative, voici les recommandations pour le choix d'une strategie de self-healing selon le contexte:

9.1 Projet academique / Recherche

Pipeline ML recommande. Il offre la meilleure couverture pour l'analyse de donnees et la comparaison de strategies. L'infrastructure Colab est gratuite et le pipeline est entierement personnalisable.

9.2 Projet professionnel - Equipe restreinte

Selecteurs natifs Playwright recommandes. Le cout de maintenance est le plus faible et la resilience est integree. Combiner avec Applitools pour la regression visuelle si le budget le permet.

9.3 Projet professionnel - Grande equipe

Combinaison: selecteurs natifs Playwright + Applitools pour les tests de regression visuelle. Les agents Copilot peuvent etre utilises pour la generation initiale des tests et le debugging.

9.4 Application legacy avec tests existants

Pipeline ML recommande pour le healing automatique des selecteurs existants. L'auto-patching permet de migrer progressivement vers des selecteurs natifs resilients.

9.5 CI/CD avec budget limite

Selecteurs natifs Playwright uniquement. Gratuits, rapides, et suffisamment resilients pour la plupart des cas d'usage.

9.6 Application critique avec UI complexe

Applitools + selecteurs natifs. La couverture visuelle combinee a la resilience DOM offre la meilleure protection.

10. Conclusion

Cette etude comparative a evalue quatre approches de self-healing pour les tests Playwright sur l'application Arcane Shop. Les resultats montrent qu'aucune approche n'est universellement superieure - le choix depend du contexte, du budget, et des objectifs.

Le pipeline ML atteint les meilleures performances quantitatives ($F1=0.97$, $Recall=100\%$) mais au cout d'une infrastructure complexe (Colab + ngrok). Les selecteurs natifs Playwright offrent la meilleure approche preventive avec un cout nul et une execution rapide, mais ne peuvent pas reparer des tests existants. Applitools complete ces approches avec une detection visuelle que les autres methodes ne peuvent pas fournir. Les agents Copilot offrent une flexibilite maximale mais restent limites par le cout et la latence des API LLM.

Recommandation finale: une strategie hybride combinant selecteurs natifs Playwright pour la prevention, pipeline ML pour le healing reactif, et Applitools pour la couverture visuelle, couvre l'ensemble des besoins de resilience des tests automatises.

11. Annexes

11.1 Methodologie de Collecte des Donnees

Les donnees du pipeline ML ont ete collectees via l'evalateur automatise (helpers/evaluator.js) qui execute chaque scenario de test contre l'API Colab et compare les resultats aux ground truth. Les metriques sont calculees par le moteur de metriques (helpers/metrics.js) qui classeifie chaque resultat en TP/FP/TN/FN et calcule accuracy, precision, recall, F1.

Les selecteurs natifs Playwright ont ete verifies via --list (158 tests enumeres). Les temps d'execution estimes sont bases sur la litterature Playwright et des tests similaires sur la meme application.

Les donnees Applitools sont basees sur les specifications du produit et les benchmarks publies par Applitools (applitools.com).

11.2 Structure des Projets

Projet 1 - self-healing-playwright-v2 (Pipeline ML):

- tests/ - 9 fichiers de test avec selecteurs casses
- helpers/fixtures.ts - Proxy Playwright avec interception des echecs
- helpers/healer.js - Client API Colab
- helpers/metrics.js - Moteur de metriques
- helpers/evaluator.js - Harness d'evaluation
- eval/scenarios/ - Scenarios de test JSON
- eval/reports/ - Rapports de metriques generes
- Rapports de healing: 40+ rapports generes

Projet 2 - playwright-native-healing (Selecteurs natifs):

- tests/ - 9 fichiers, 158 tests, selecteurs resilients
- helpers.ts - waitForProducts et BASE_URL
- playwright.config.ts - Configuration standard Playwright

Projet 3 - playwright-applitools (Applitools):

- tests/ - 9 fichiers avec checkpoints visuels
- fixtures.ts - Fixture custom avec integration Eyes
- helpers.ts - waitForProducts et BASE_URL

Projet 4 - playwright-agents-demo (Agents Copilot):

- .github/agents/ - 3 agents (planner, generator, healer)
- tests/ - 7 fichiers, 29 tests TodoMVC
- specs/ - Plans de test en markdown

11.3 Environnement Technique

- Playwright v1.60.0
- Node.js 20+ / TypeScript 5.9
- Python 3.10+ (Colab)
- all-MiniLM-L6-v2 (Sentence-Transformers)
- EfficientNet-B0 (Computer Vision)
- SSIM (Structural Similarity Index)
- TF-IDF Vectorizer (scikit-learn)
- Applitools Eyes SDK v1.29+
- Ngrok tunnels pour exposition API

11.4 Glossaire

TP / FP / TN / FN:	True Positive, False Positive, True Negative, False Negative
F1 Score:	Moyenne harmonique de precision et recall
SSIM:	Structural Similarity Index Measure
CNN:	Convolutional Neural Network
TF-IDF:	Term Frequency - Inverse Document Frequency
MCP:	Model Context Protocol (outils LLM)
ngrok:	Tunnel securise vers un serveur local

Colab:

Google Colaboratory (notebook cloud gratuit)